

JAVA OOP PRACTICAL RECORD

Experiment No. 06

Experiment No.	06
Date	23/09/2026
Title	Longest Substring
Concept	Strings, Nested Loops and Character Checking

1. Objective / Aim

To write a Java program that accepts a string and finds the longest substring without repeating characters using nested loops and String methods.

2. System Requirements

Hardware: Standard PC/Laptop with 4 GB/8 GB RAM and a Core i3/i5 processor or equivalent.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, and a Java editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Create the LongestSubstring class and a Scanner object.
2. Read a string from the user using `nextLine()`.
3. Initialize `maxLength` to 0 and `longest` to an empty string.
4. Use an outer loop to choose each possible starting position.
5. For each starting position, initialize `current` as an empty string.
6. Use an inner loop to examine characters one by one.
7. Use `indexOf()` to check whether the current character already exists in `current`.
8. If a repeated character is found, stop the current substring.
9. Compare `current.length()` with `maxLength` and update `longest` when a longer substring is found.
10. Display the length and contents of the longest substring.
11. Close the Scanner.

4. UML Class Diagram

LongestSubstring
- sc : Scanner
- str : String
- maxLength : int
- longest : String
- current : String
- ch : char
+ main(args : String[]) : void

The program contains one user-defined class, LongestSubstring, which uses the Java Scanner class for input.

5. Source Code

```
import java.util.Scanner;

public class LongestSubstring {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        int maxLength = 0;
        String longest = "";

        for (int i = 0; i < str.length(); i++) {
            String current = "";

            for (int j = i; j < str.length(); j++) {
                char ch = str.charAt(j);

                // Check whether character is already present
                if (current.indexOf(ch) != -1) {
                    break;
                }

                current += ch;
            }

            if (current.length() > maxLength) {
                maxLength = current.length();
                longest = current;
            }
        }

        System.out.println("Length of longest substring: " + maxLength);
        System.out.println("Longest substring: " + longest);

        sc.close();
    }
}
```

6. Compilation & Execution Commands

```
javac LongestSubstring.java
java LongestSubstring
```

7. Sample Output Screen

```
Enter a string: abcabcbb
Length of longest substring: 3
Longest substring: abc
```

8. Result

The LongestSubstring program was successfully implemented. It finds and displays the longest substring that contains no repeated characters.